Vulkan migration: executive assessment

Easy3D repository audit

2026-11-09

Audit date: 2026-11-09 **Audited revision:** `9802ed2f` (`Improve API documentation cross-references`)
Scope: the current repository tree, `docs/vulkan.md`, `ToDo.md`, and the OpenGL/Vulkan viewer, renderer, GUI, build, and test structure.

1 Executive assessment

The migration has reached broad functional coverage for the shared Easy3D viewer and tutorial workflows, but "complete" needs a qualifier. The core viewer, `MultiViewer`, `OffScreen`, renderer-backed drawing, picking, selection, snapshots, overlays, ImGui, and the majority of tutorials are now available in either configured backend. The two backends are selected at build time; they are not linked together in one build. Mapple is an explicit OpenGL-only exception, Qt/wx viewer tutorials were removed by decision, and the Python scope is deliberately limited to the supported backend-neutral API.

OpenGL remains the better **behavioral reference and more integrated implementation** today. It is the established backend and retains the direct, mature Easy3D rendering path. Vulkan has the better **explicit low-level resource and command model**: immutable CPU descriptions, explicit device ownership, validation, transitions, submissions, and readback make its operations easier to reason about than implicit context-global GL state. But Vulkan is not yet the better overall implementation: its backend is concentrated in a very large `render_device.cpp`, frame work is synchronous, resource capacity and allocation strategies are fixed, and platform validation is incomplete. Neither backend is uniformly superior.

The migration is substantially more aligned than the earlier assessment: many previously open features have since landed, including `MultiViewer`, common GUI pickers, GPU selection, effects, order-independent transparency, effect-aware snapshots, and all algorithm tutorials. The remaining concern is less "missing basic feature" and more **proving equivalence, avoiding API surprises, and keeping the Vulkan implementation maintainable and scalable**.

2 Current status and evidence

The tracker records the following Mac test results after batch 44 (2026-11-08): OpenGL 105/105, MoltenVK 139/139, OpenGL+Qt 104/104, and Qt/CGAL/ffmpeg Vulkan Debug 144/144. It also reports load-sensitive tests that fail under concurrent `-j2` runs but pass alone. These are recorded results, not a fresh rerun for this assessment. This worktree contains the configured Vulkan Debug tree with 144 registered tests; the other named build trees are not present here.

The tracker also leaves two concrete validation items open:

- The earlier Linux Vulkan baseline was 68/69: `vulkan.interactionoverlay` disagreed with its one-pixel border expectation. The latest tracker still says this needs resolution against the relevant Mesa/CI environment.
- The portable expanded-point fallback has not been profiled on a MoltenVK device without the optional `largePoints` feature. The measured native path was about 80 FPS at 3200×1834, 4× MSAA, with zero validation errors; this is not a Vulkan-versus-OpenGL comparison and says nothing about the fallback.

Windows hardware execution and cross-platform packaging are not established by the Mac results. The checkout has no active `.github/workflows/vulkan.yml`; the workflow file is contained in `.github/uncompress_here` Documentation should not claim a running CI matrix until an actual workflow file is restored and its runs are observed.

3 Alignment and quality by area

Backend selection and package structure Good: one backend per build through `Easy3D_ENABLE_VULKAN`, no runtime fallback, and shared class names. Separate builds need continued installed-consumer checks.

Viewer and tutorial workflow Broadly aligned: GLFW viewer, callbacks, input, camera operations, overlays, ImGui and the migrated tutorials use a shared surface. Mapple remains intentionally OpenGL-only; Qt/wx viewer tutorials were removed, not ported.

Public API compatibility Not identical in every detail. The constructor was intentionally reordered and the default sample count changed; old positional calls can still compile but mean something different. In addition, `viewer.h` exposes Vulkan-only operations inside `#ifdef EASY3D_HAS_VULKAN` (for example `device()`, `set_clipping_`
`scan_visible_points()` and a Vulkan-specific `add_model` overload). The common API works across builds, but clients using these extensions need conditional code.

Rendering and drawable semantics Strong and much improved: geometry, colors, textures, material state, point/line styles, clipping, MSAA, highlighting and coplanar-edge handling have substantial counterpart work. Pixel tests cover selected mesh, texture, vertex-color and drawable-state cases. The Vulkan adapter translates Easy3D drawables into explicit commands; OpenGL still uses its legacy direct rendering path, so implementation parity is maintained in two different execution models.

Picking and selection Strong functional coverage: model, surface-mesh and point-cloud picker counterparts exist; Vulkan and OpenGL use GPU-backed visible-surface/region paths. Raw `FrameImage` primitive provenance is still not universally available on OpenGL, so callers should distinguish the shared picker result from raw capture IDs.

Effects and transparency Broad parity, with known algorithmic boundaries. GTAO, EDL and hard/soft shadows are implemented on both; Vulkan shadow kernels use fewer taps than OpenGL (32/64 versus 64/128 PCSS blocker/filter taps). `AverageColorBlending` and peeling have distinct Vulkan composites. Dual-depth peeling can still show intersection artifacts; per-pixel linked lists addresses quality at a substantial memory/performance cost and can drop fragments when its configured pool overflows.

Snapshots and coordinates Much improved: shared capture contracts, logical-window versus framebuffer conventions, DPI-aware pickers and effect-aware snapshots are documented and tested. OpenGL now avoids effect seams for in-limit images by rendering one whole-image FBO. Oversized-image and device-limit behavior still needs platform coverage.

Resource correctness and diagnostics Vulkan's explicit ownership, barriers, capability checks, initialization errors and validation are a strength. OpenGL remains simpler but depends on current-context discipline and mutable global state; the OpenGL query-state guard and recent snapshot/HUD fixes reduce, but do not eliminate, that class of risk.

Maintainability and extension OpenGL remains easier to navigate for ordinary renderer changes because work is distributed among established drawable/effect classes and shaders. Vulkan's command boundary is a useful extension seam, but its implementation is unusually centralized and its `DrawCommand` combines many render modes and constraints. The backend is likely to become harder to extend without splitting internal responsibilities and strengthening typed pass contracts.

Throughput and scale Not proven equivalent. Vulkan currently submits synchronously and waits on a fence, with one frame in flight; it uses individual allocations, fixed descriptor capacity (1024 live textures), and a large per-pixel linked-list allocation when that OIT mode is selected. These choices favor correctness and simple lifetime reasoning over throughput or large-scene scalability. No controlled, same-scene Vulkan/OpenGL benchmark is recorded.

4 Findings

4.1 1. Treat the migration as broad feature parity, not proven output parity

The tracker records extensive counterpart implementation and separate full-suite results, but the normal build selects only one backend. Passing an OpenGL suite and a Vulkan suite does not prove that the same inputs produce equivalent output or interactions. The pixel-level tests are valuable, but their scope is selective rather than a systematic backend comparison.

Recommendation: add or restore a small deterministic cross-backend comparison harness that runs the same scenes and probes in separate build trees, then compares images, depth, IDs, resize/DPI behavior, picker results, and effects with documented tolerances. Keep exact invariants (IDs, geometry, event semantics) separate from expected approximation differences (shadow taps, implementation-specific AA).

4.2 2. Make the intentional constructor break hard to misuse

The documented `Viewer` constructor differs from the original OpenGL argument order and default MSAA behavior. An old call with positional integers may compile while interpreting former sample/context arguments as width, height, and sample count. This is a silent runtime-behavior change, not a compile-time migration signal.

Recommendation: before declaring source compatibility, decide whether to retain a deprecated, unambiguous compatibility overload or require named/options-based construction for customized settings.

At minimum, give migration examples and compile tests for old call shapes, and document the silent reinterpretation prominently. This is a user-facing compatibility decision, not a backend defect that can be solved by matching renderer output.

4.3 3. Preserve the Vulkan explicitness; reduce its concentration

`easy3d/renderer/render_device.h` provides a useful backend-neutral geometry/image/command contract. The Vulkan implementation in `easy3d/renderer/backends/vulkan/render_device.cpp` owns context/device creation, caches, descriptors, render passes, effects, OIT, readback and swapchain presentation in one large translation unit and a large device class. `viewer.cpp` also contains separately compiled backend implementations, including duplicated viewer interaction/frame logic. This makes changes expensive to review and creates opportunities for behavior to drift between branches.

Recommendation: keep the public `RenderDevice` and viewer contracts stable, but separate Vulkan internals along resource ownership and lifecycle boundaries (device/context and swapchain; geometry/texture realization; frame targets and passes; effects/OIT; capture/readback). First add focused lifetime and synchronization tests around those boundaries. Avoid a broad abstraction rewrite: the current explicit command model is useful, but a second generic framework would add indirection without proving value.

4.4 4. Improve Vulkan throughput only against a measured baseline

The current synchronous fence wait keeps resource lifetime and readback straightforward, but prevents useful CPU/GPU overlap. Per-resource allocations and fixed descriptor budgets also impose scalability ceilings. The per-pixel linked-list transparency mode is intentionally expensive and its capacity is a hard correctness/quality limit.

Recommendation: first record reproducible OpenGL/Vulkan frame-time and memory baselines for representative mesh, point-cloud, effect-heavy and snapshot workloads. Then consider a small frames-in-flight ring, deferred destruction by submission serial, staging reuse, and suballocation. Keep synchronous capture as an explicit API if deterministic readback remains useful. Add overflow diagnostics or a visible overflow count for linked-list OIT rather than relying only on a user-chosen capacity.

4.5 5. Finish external validation and make CI claims actionable

The latest tracker reports strong Mac coverage but one unresolved Linux overlay assertion and no Windows hardware result. Several tests are load-sensitive. The intended Vulkan CI workflow is archived rather than located under GitHub Actions' active workflow directory in this checkout.

Recommendation: restore the intended workflow, run independent backend jobs with bounded concurrency, build and test at least one supported Windows Vulkan device and Linux hardware/software Vulkan path, and retain logs for the current overlay mismatch. Mark a flaky test as resolved only after isolated and full-suite runs are stable on the supported runner.

4.6 6. Keep the remaining differences explicit and bounded

The migration has made major progress on effects and GUI picking, so old "pending" claims about those features are obsolete. Remaining boundaries are narrower: Mapple is an explicit OpenGL-only application; Qt/wx viewer tutorials are retired; advanced text shaping/bidi/fallback, line joins/rounded caps,

automatic clipping caps, PBR-grade material behavior and per-drawable AO are not established as parity goals. Resource and presentation constraints (device formats, queue-family requirements, allocation limits) must remain documented as Vulkan platform constraints.

Recommendation: separate migration blockers from deliberate product-scope decisions and optional quality backlog. Do not reopen Mapple or Qt/wx porting unless requirements change. For each retained boundary, state whether it is an intentional unsupported feature, an approximation, or a platform-dependent failure.

5 Overall comparison

Better today for established Easy3D behavior and day-to-day maintainability: OpenGL. It is the mature implementation and still carries the most direct mapping to the original renderer and toolkit application. Its weakness is implicit context/state dependence and harder-to-audit synchronization/lifetime behavior.

Better foundation for explicit GPU resource and command semantics: Vulkan. Its immutable inputs, explicit capabilities, validation and ownership are strong design choices, and they make unsupported conditions more diagnosable. Its weakness is implementation concentration, serialized submission, fixed resource budgets, and incomplete platform evidence.

The recommended direction is not to "replace the better backend with the worse one." Preserve OpenGL as the behavioral oracle where verified, preserve Vulkan's explicit resource model, and improve both through shared semantic tests. Make OpenGL state/context requirements more locally managed; make Vulkan internals smaller, measured and asynchronous only where profiling justifies it.

6 Remaining work, in order

1. **Close the known Linux Vulkan failure:** reproduce `vulkan.interactionoverlay` on the intended Mesa/CI version, decide whether the code or assertion is wrong, then run it repeatedly and in the full suite.
2. **Restore and validate active CI:** move the intended workflow from the archived `.github/uncompress_here.zip` into `.github/workflows/`; run Linux Vulkan/OpenGL jobs and capture validation output.
3. **Complete platform coverage:** run Vulkan on Windows hardware and verify install/consumer packaging on Windows and Linux; retain the existing macOS/MoltenVK matrix.
4. **Profile the untested fallback:** exercise and benchmark the RealCamera expanded-point fallback on MoltenVK without `largePoints`, and collect a controlled OpenGL/Vulkan comparison at matched resolution, sample count and scene.
5. **Resolve the public-API compatibility risk:** document/mitigate the constructor change and publish the exact common API versus compile-time backend extensions.
6. **Create systematic parity tests:** compare representative image, depth, ID, picking, interaction, snapshot and DPI outputs across the two separately configured builds.
7. **Then improve maintainability/performance:** modularize Vulkan internals and profile resource allocation/frame scheduling before adopting a frames-in-flight or allocator redesign.
8. **Track optional feature gaps separately:** text shaping, line joins/caps, clipping caps, PBR/material refinements and similar features should not be confused with the core migration acceptance gate unless the supported API promises them.

7 Source references

- `docs/vulkan.md` — current architecture, backend policy, coordinate conventions, capability boundaries and implementation history.
- `ToDo.md` — batch 44 status, Nov 8 Mac test counts, load-sensitive failures, Linux overlay status, fallback profiling, and the Mapple/Python scope decisions.
- `easy3d/viewer/viewer.h`, `viewer.cpp`, `multi_viewer.cpp`, `offscreen.cpp` — common viewer surface and compile-time backend implementations.
- `easy3d/renderer/render_device.h` and `easy3d/renderer/backends/vulkan/render_device.cpp` — immutable command contract, explicit Vulkan device/resource/pipeline implementation, synchronous submission, and fixed descriptor limits.
- `easy3d/renderer/backends/opengl/`, `easy3d/renderer/*.cpp`, and renderer shader sources — legacy GL execution path and its stateful context requirements.
- `tests/` — separate configured-backend tests; useful focused raster checks, but not a comprehensive cross-backend image-diff suite.